

国家级精品课程
山东省一流本科课程



程序设计基础（C语言）

信息科学与工程学院

潘玉奇

第7章 结构体

- ◆ 7.1 结构体的引出
- ◆ 7.2 结构体变量
- ◆ 7.3 结构体数组
- ◆ 7.4 结构体类型的指针变量
- ◆ 7.5 结构体与函数
- ◆ 7.6 链 表

7.1 结构体的引出

【问题描述7.1】学生信息包括：学号、姓名、年龄、3门课的成绩和总成绩，编程实现以下功能：(1) 输入3个学生的信息；(2) 计算每个学生的总成绩并输出。

分析： 首先可以把学生信息用表格的形式表现出来

学号	姓名	年龄	英语	数学	C语言	总成绩
2009001	赵岩	18	86	90	82	258
2009002	王洋	19	92	88	89	269
2009003	李玲	18	80	85	79	244

根据已有知识，我们可能需要分别定义以下5个数组：

```
char num[3][16];    //学号用二维字符数组存放
char name[3][12];   //姓名用二维字符数组存放
int age[3];          //年龄用一维整型数组存放
int score[3][3];     //3门课成绩用二维整型数组存放
int total[3];         //总成绩用一维整型数组存放
```

这种数据存放方式不太理想

7.1 结构体的引出

➤ 用多个数组解决时的内存分配情况如下

2009001	赵岩	18	86	90	82	258
2009002	王洋	19	92	88	89	269
2009003	李玲	18	80	85	79	244

➤ 我们希望的内存分配情况

2009001	2009002	2009003
赵岩	王洋	李玲
18	19	18
86	92	80
90	88	85
82	89	79
258	269	244

分配内存不集中，寻址效率不高；对数组进行赋初值时，容易发生错位；结构显得比较零散，不容易管理

7.1 结构体的引出

为解决问题，C语言给出一种构造数据类型——**结构体类型**。

- ◆ 结构体是由若干“**成员**”组成的，每一个成员可以是一个**基本数据类型**或者又是一个**构造类型**。

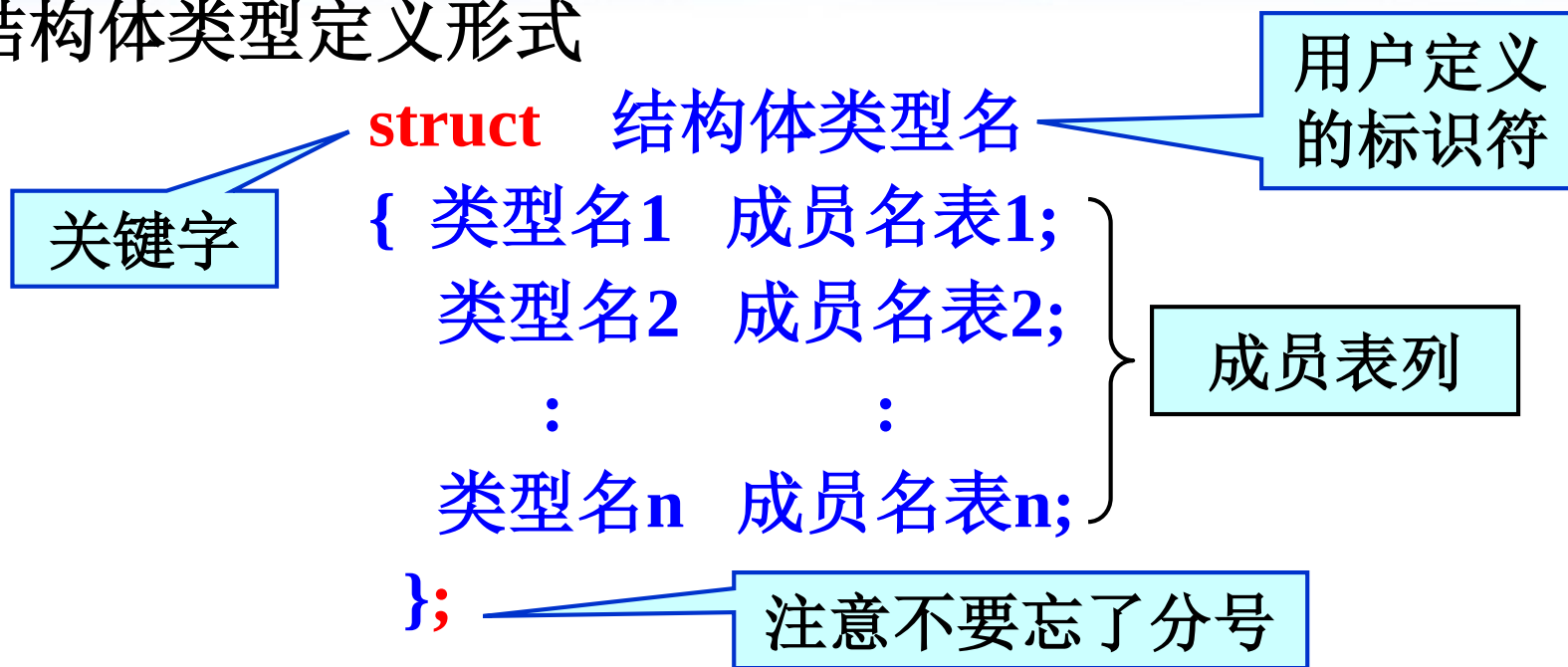
对于上述学生信息，我们可以定义一个结构体类型，它包含5个成员，每个成员的数据类型如下：

学号	姓名	年龄	3门课成绩	总成绩
char num[16]	char name[12]	int age	int score[3]	int total



7.1 结构体的引出

◆ 结构体类型定义形式



说明:

- ① 结构体成员的类型可以是基本类型、数组、指针或已说明过的**结构体类型**等。
- ② 每个“成员名表”都可以包含多个相同类型的成员名，它们之间用逗号隔开。

7.1 结构体的引出

◆ 学生结构体类型定义

学号	姓名	年龄	3门课成绩	总成绩
char num[16]	char name[12]	int age	int score[3]	int total

```
struct student1
{ char num[16];
  char name[12];
  int age;
  int score[3];
  int total;
};
```

```
struct date
{ unsigned int year;
  unsigned int month;
  unsigned int day;
};
```

定义一个日期
结构体类型

```
struct student2
{ char num[16];
  char name[12];
  struct date birthday;
  int score[3];
  int total;
};
```

结构体类型
嵌套定义

7.2 结构体变量

7.2.1 结构体变量的定义

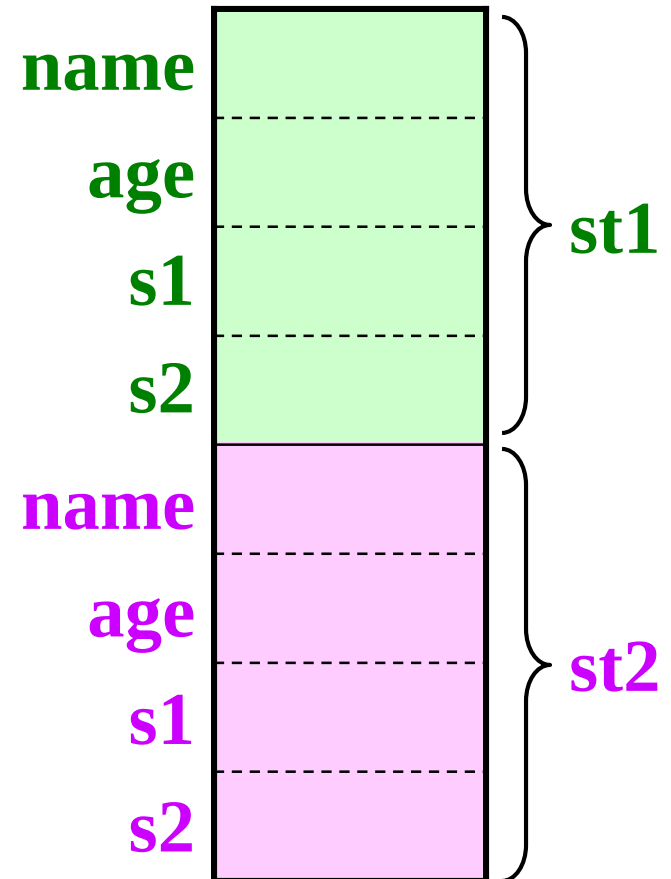
1. 先定义结构体类型, 再定义结构体变量

```
struct student  
{ char name[12];  
  int age;  
  int s1, s2;  
};
```

结构体
类型定义

```
struct student st1, st2;
```

结构体
变量定义



7.2.1 结构体变量的定义

- ◆补充：在内存中结构体变量占有一片连续的存储单元，其占用的字节数可用**sizeof** 运算符算出

```
struct student
{ char  name[12];
  int  age;
  int  s1, s2 ;
};
```

```
struct student  st1;
```

```
printf(“%d\n”, sizeof(struct  student) );
```

```
printf(“%d\n”, sizeof(st1) );
```

sizeof的使用:

```
int  a,b,c;
a=sizeof(int);
b=sizeof(float);
c=sizeof(char);
```

7.2.1 结构体变量的定义

2. 在定义结构体类型的同时定义结构体变量

```
struct student
```

```
{ char name[12];
```

```
  int age;
```

```
  int s1, s2;
```

```
} st1, st2;
```

结构体变量定义

3. 直接定义结构体变量

```
struct
```

```
{ char name[12];
```

```
  int age;
```

```
  int s1, s2;
```

```
} st1, st2;
```

注意: 这里没有结构体类型名, 这种方式有时使用并不方便, 因此不建议大家采用

7.2.2 结构体的引用和初始化

1. 结构体变量的引用

一般对结构体变量的使用，如赋值、输入、输出、运算等都是对结构体变量的**成员**进行操作。

结构体变量成员的表示格式：**结构体变量名.成员名**

说明：圆点“.”称为结构体成员运算符，它的运算优先级是最高的。可以把“结构体变量名.成员名”看成一个整体，像简单变量一样使用。

```
struct student
{ char name[12];
  int age;
  int s1, s2;
};
```

```
struct student st1;
strcpy(st1.name, "Mary");
st1.age = 21;
st1.s1 = 78;
st1.s2 = 86;
```

注意：一般是对结构体变量的各个成员分别进行赋值
`st1 = {"Mary", 21, 78, 86};` 这样的赋值是不允许的



7.2.2 结构体的引用和初始化

1. 结构体变量的引用

对于多层嵌套结构体成员的使用，应按照**从最外层到最内层的顺序逐层使用成员名**，每层成员名之间用圆点隔开，只能对最内层的成员进行存取、运算及输入、输出等操作。

```
struct date
{ unsigned int year;
  unsigned int month;
  unsigned int day;
};

struct stud
{ char name[12];
  struct date birthday;
  int s1, s2;
};
```

```
struct stud st2;
strcpy(st2.name, "John");
st2.birthday.year = 1990;
st2.birthday.month = 11;
st2.birthday.day = 23;
st2.s1 = 89;
st2.s2 = 95;
```



7.2.2 结构体的引用和初始化

【例7.1】 对结构体变量赋值并输出其值。

```
#include<stdio.h>
#include<string.h>
struct date
{  unsigned int year;
    unsigned int month;
    unsigned int day;
};
struct stud
{  char num[16];
    char name[12];
    struct date birthday;
    int score;
};

int main()
{
    struct stud a;
    strcpy(a.num, "20131234001");
    strcpy(a.name, "赵岩");
    a.birthday.year=1994;
    a.birthday.month=11;
    a.birthday.day=25;
    a.score=85
    printf("学号: %s\n姓名: %s\n", a.num, a.name);
    printf("出生日期: %d.%d.%d\n ", a.birthday.year,
        a.birthday.month, a.birthday.day);
    printf("成绩: %d\n", a.score);
    return 0;
}
```



7.2.2 结构体的引用和初始化

2. 结构体变量的初始化

```
struct student
{ char name[12];
  int age;
  int s1, s2 ;
} st1={"Mary", 21, 78, 86};
```

初始化, 正确

```
struct stud
{ char name[12];
  struct date birthday;
  int s1, s2 ;
};
```

初始化, 正确

```
struct stud st2={"John", 1980, 11, 23, 89, 95 } ;
```

```
struct student
{ char name[12];
  int age;
  int s1, s2;
};
struct student st1;
st1={"Mary", 21, 78, 86} ;
```

这是赋值, 错误
C不允许这么做

7.2.2 结构体的引用和初始化

3. 相同类型结构体变量的“整体”赋值

```
struct date
{ unsigned int year;
  unsigned int month;
  unsigned int day;
};
struct stud
{ char name[12];
  struct date birthday;
  int s1, s2;
};
```

```
struct stud st1, st2, st3;
strcpy(st1.name, "John");
st1.birthday.year = 1990;
st1.birthday.month = 11;
st1.birthday.day = 23;
st1.s1 = 89;
st1.s2 = 95;
st2=st1;
strcpy(st3.name, "Mary");
st3.birthday=st1.birthday;
st3.s1 = 76;
st3.s2 = 85;
```

注意要正确赋值的条件是变量st1已经有了数据

7.2.2 结构体的引用和初始化

3. 相同类型结构体变量的“整体”赋值

【例7.2】对结构体变量进行初始化。

```
#include<stdio.h>
```

```
struct date
```

```
{ unsigned int year;
```

```
    unsigned int month;
```

```
    unsigned int day;
```

```
};
```

```
struct person
```

```
{ char name[12];
```

```
    struct date birthday;
```

```
    double wage;
```

```
};
```

//定义职员类型

//姓名

//出生日期

//工资

7.2.2 结构体的引用和初始化

【例7.2】对结构体变量进行初始化。

```
int main( )
{   struct person a={ "李立", 1975, 8, 12, 3256.78 };
    struct person b={ "张红" };
    printf(" 姓名: %s, ", b.name);
    printf(" 出生日期: %d.%d.%d, " , b.birthdate.year,
                                                b.birthdate.month, b.birthdate.day);
    printf(" 工资: %8.2lf\n ", b.wage);
    b.birthdate=a.birthdate; //结构体成员进行整体赋值
    b.wage=4786.45;
    printf(" 姓名: %s, ", b.name);
    printf(" 出生日期: %d.%d.%d, " , b.birthdate.year,
                                                b.birthdate.month, b.birthdate.day);
    printf(" 工资: %8.2lf\n ", b.wage);
    return 0;
}
```

输出: 姓名: 张红, 出生日期: 0.0.0, 工资: 0.00 786.45



7.2.2 结构体的引用和初始化

【问题描述7.1】 学生信息包括：学号、姓名、年龄、3门课的成绩和总成绩，编程实现功能：(1) 输入3个学生的信息；(2) 计算每个学生的总成绩并输出。

```
#include<stdio.h>
```

```
struct student      //定义结构体类型
```

```
{ char  num[16];
```

```
  char  name[12];
```

```
  int  age;
```

```
  int  score[3];
```

```
  int  total;
```

```
};
```

```
int main( )
```

```
{ struct student  st1, st2, st3;      //定义3个结构体变量存放学生信息
```

```
  int  i, sum;
```

```
  printf("\n输入学生1的信息:\n");
```

```
  scanf(" %s%s%d", st1.num, st1.name, &st1.age);
```

```
  for(i=0; i<3; i++)
```

```
      scanf(" %d", &st1.score[i]);
```



7.2.2 结构体的引用和初始化

【问题描述7.1】 参考程序

```
int main( )
{
    .....    // 学生2、3的信息输入略

    for( i=0, sum=0; i<3; i++)
        sum=sum+st1.score[i];    //计算学生1的总成绩
    st1.total=sum;
    printf("学生1的总成绩=%-4d\n", st1.total);
    .....    // 学生2、3的总成绩计算和输出略
    return 0;
}
```

7.2.2 结构体的引用和初始化

◆ 注意

(1) 结构体类型和结构体变量是不同的概念

```
struct student  
{   char  name[12];  
    int   age;  
    int   s1, s2 ;  
};
```

```
struct student st1;  
student . age=20;
```

错！ student是结构体类型的名字，并不是结构体变量

```
st1. s1=98;  
st1. s2=87;
```

对！ st1是结构体变量，可以对其成员进行赋值

7.2.2 结构体的引用和初始化

◆ 注意

(2)只能对结构体变量的成员进行输入和输出操作

```
struct student  
{  char  name[12];  
    int  age;  
    int  s1, s2 ;  
};
```

```
struct student  st1;
```

```
scanf("%s%d%d%d", &st1);
```

```
scanf("%s%d", st1.name, &st1.age);
```

错！ 想对结构体变量整体进行输入是行不通的

对！ 只能对结构体变量的成员进行输入

7.3 结构体数组

7.3.1 结构体数组的定义

1. 先定义结构体类型
再定义结构体数组

```
struct student  
{ char name[12];  
  int age;  
  int s1, s2;  
};  
struct student st[10];
```

2. 定义结构体类型同时定义数组

```
struct student  
{ char name[12];  
  int age;  
  int s1, s2 ;  
} st[10];
```

3. 直接定义结构体数组

```
struct  
{ char name[12];  
  int age;  
  int s1, s2 ;  
} st[10];
```

7.3.2 结构体数组的初始化

- 结构体数组初始化的方法与数组的初始化相类似。
由于每个数组元素都是一个结构体类型的数据，可以将每个元素中成员的初值依次放在花括号{ }内，以便区分各个元素。

```
struct stu
{
    char num[16];
    char name[12];
    int score;
};
```

```
struct stu st[3]={ {"2009001", "赵岩", 86 },
                   {"2009002", "王洋", 92 },
                   {"2009003", "李玲", 78 } };
```

2009001	}	st[0]
赵岩		
86		
2009002	}	st[1]
王洋		
92		
2009003	}	st[2]
李玲		
78		

7.3.2 结构体数组的初始化

◆ 关于结构体数组初始化的说明

① 如果对全部数组元素赋初值，则可以不给出数组长度。

```
struct stu st[ ]={ {"200901", "赵岩", 86}, {"200902", "王洋", 92} };
```

② 可以只对数组中的某些元素赋初始值。

```
struct stu st[3]={ {"200901", "赵岩", 86}, {"200902", "王洋", 92} };
```

```
struct stu st[3]={ {"200901", "赵岩", 86}, {0}, {"200903", "李玲", 78}};
```

③ 初始化时也可以省略内层的花括号。

```
struct studt st[3]={"200901", "赵岩", 86, "200902", "王洋", 92, "200903", "李玲", 78 };
```

这样写各元素的初值连成一片，容易混淆，
所以建议一般不要省略内层的花括号。

7.3.3 结构体数组的使用

◆ 一个结构体数组的元素相当于一个结构体变量。

(1) 引用某个数组元素的成员

```
strcpy(st[0].num, "200901");  
puts( st[1]. name );
```

(2) 数组元素之间可以整体赋值,
也可以将一个数组元素赋给
一个相同类型的结构体变量

```
struct stu x;
```

```
st[2] = st[0];
```

```
x = st[1];
```

} 这都是结构体变量的整体赋值形式

```
struct stu  
{ char num[16];  
  char name[12];  
  int score;  
} st[3]={ ...略 };
```

(3) 只能对数组元素的**成员**进行输入和输出

7.3.3 结构体数组的使用

【例7.3】设有学生40人，学生信息包括：学号、姓名、3门课的成绩和总成绩，编程实现以下功能：(1) 输入学生的信息；(2) 计算每个学生的总成绩；(3) 输出总成绩高于270分的学生信息。

分析：一个学生的信息用一个结构体变量来存放，40个学生的信息用结构体数组存放，每个数组元素存放一个学生的信息。

```
#include<stdio.h>
#define N 40          //定义符号常量N，表示学生人数
struct student        //在函数外定义结构体类型
{
    char num[16];
    char name[12];
    int score[3];
    int total;
};
```



7.3.3 结构体数组的使用

【例7.3】方法1：分别用3个for循环实现题目要求的3个功能

```
int main( )  
{ struct student st[N];    //定义结构体数组  
  int i, j, sum;  
  for(i=0; i<N; i++)  
  { scanf("%s%s", st[i].num, st[i].name );  
    for(j=0; j<3; j++)  
      scanf("%d", &st[i].score[j]);  
  }  
  for(i=0; i<N; i++)    //计算每个学生的总成绩  
  { sum=0;  
    for( j=0; j<3; j++)  
      sum=sum+st[i].score[j];  
    st[i].total=sum;  
  }
```

... 见下页



7.3.3 结构体数组的使用

【例7.3】方法1： 分别用3个for循环实现题目要求的3个功能

```
int main( )
{
    ... 接上页
    printf("\n输出总成绩高于270分的学生信息:\n");
    for(i=0; i<N; i++)
    {
        if ( st[i].total>270 )
        {   printf(" %-16s %-12s ", st[i].num, st[i].name);
            for(j=0; j<3; j++)
                printf(" %-3d ", st[i].score[j]);
            printf(" %-4d\n", st[i].total);
        }
    }
    return 0;
}
```



【例7.3】方法2：将前2个for循环合并成一个for循环

```
int main( )
```

```
{ struct student st[N];      int i, j, sum;
```

```
  for(i=0; i<N; i++)
```

```
  {   scanf("%s%s", st[i].num, st[i].name );
```

```
    sum=0;
```

```
    for( j=0; j<3; j++ )
```

```
    {   scanf("%d", &st[i].score[j]);
```

```
        sum=sum+st[i].score[j];
```

```
    }
```

```
    st[i].total=sum;
```

```
  }
```

```
  for(i=0; i<N; i++)
```

```
  {   if ( st[i].total>270 )
```

```
    {   printf("%-16s %-12s ", st[i].num, st[i].name);
```

```
        for(j=0; j<3; j++) { printf("%-3d ", st[i].score[j]); }
```

```
        printf("%-4d\n", st[i].total);
```

```
    }
```

```
  }
```

```
  return 0;
```

```
}
```

7.4 结构体类型的指针变量

7.4.1 指向结构体变量的指针

◆ 结构体指针变量的定义形式

struct 结构体类型名 *结构体指针变量名

例：

```
struct stu
{   char num[16];
    char name[12];
    int score;
};
struct stu a,*p;
p=&a;
```

通过指针变量，可以访问结构体变量的各个成员，方法如下：

- ① **(*结构体指针变量名). 成员名**
(*p).score=89;
- ② 结构体指针变量名 **->** 成员名
p->score=89;

注意：结构体指针变量在使用前必须先对其进行赋值

7.4.1 指向结构体变量的指针

【例7.4】 结构体指针变量的使用。

```
#include<stdio.h>

struct stu
{   char num[20];
    char name[20];
    int score;
};

int main()
{   struct stu  boy1={ "20131234001","赵岩", 86}, *p;
    p=&boy1;
    printf("学号=%s\n", boy1.num);
    printf("姓名=%s\n", (*p).name);
    printf("成绩=%d\n", p->score );
    return 0;
}
```



7.4.2 指向结构体数组的指针

➤ 结构体指针变量可以指向一个结构体数组，这时结构体指针变量的值是结构体数组的首地址。

例如： **p=a;**

➤ 结构体指针变量也可指向结构体数组中的一个元素，这时结构体指针变量的值是该数组元素的地址。

例如： **p=&a[5];**

```
struct stu
{   char num[16];
    char name[12];
    int score;
};
struct stu a[10],*p;
```

7.4.2 指向结构体数组的指针

【例7.5】用指向结构体数组的指针变量输出数组中数据。

```
#include<stdio.h>
```

```
struct stu
```

```
{  char num[20];
```

```
    char name[20];
```

```
    int score;
```

```
} st[4]= { { "20131234001", "赵岩", 86},  
           { "20131234002", "王洋", 92},  
           { "20131234003", "李玲", 78},  
           { "20131234004", "张强", 88}  };
```

```
int main( )
```

```
{  struct stu *ps;
```

```
    for( ps=st; ps<st+4; ps++ )
```

```
        printf(" %s, %s, %d\n", ps->num, ps->name, ps->score);
```

```
    return 0;
```

```
}
```



7.5 结构体与函数

7.5.1 结构体变量作函数参数

函数的实参与形参是同一种类型的结构体变量，函数调用时，将实参的值传给形参，在函数执行过程中，形参的任何变化都不会影响实参的值。

【例7.6】编程统计不及格的学生人数。

方法1：编写一个函数判断成绩是否及格，用结构体变量作函数参数，在main函数中调用该函数，并统计不及格的人数

```
#include<stdio.h>
#define N 10
struct stu
{  char num[20];
   char name[20];
   int score;
};
```



7.5.1 结构体变量作函数参数

【例7.6】 参考程序，方法1

int judge(struct stu x) //函数定义，结构体变量作参数

```
{  if ( x.score<60 )  return 1;
    else  retrun 0;
}
```

int main()

```
{  struct stu st[N];    int i, n=0;
```

```
    printf("请输入%d个学生的信息: \n", N);
```

```
    for(i=0; i<N; i++)
```

```
        scanf(" %s%s%d", st[i].num, st[i].name, &st[i].score);
```

```
    for(i=0; i<N; i++)
```

```
        if ( judge(st[i]) ) //函数调用，将元素st[i]的值传给形参变量x
```

```
            n++;
```

```
    printf("不及格的学生人数为:  %d\n", n);
```

```
    return 0;
```

```
}
```



7.5.2 指向结构体变量的指针作函数参数

函数调用时，若结构体指针变量(或结构体数组名)作函数实参，则形参实际上得到的是一个地址，此时在函数执行过程中，形参的任何变化都会影响到实参。

【例7.6】编程统计不及格的学生人数。

方法2：编写一个函数统计不及格的人数，用结构体数组作函数参数

```
#include<stdio.h>
#define N 10
struct stu
{   char num[20];
    char name[20];
    int score;
};
```



7.5.2 指向结构体变量的指针作函数参数

【例7.6】参考程序，方法2

```
int count(struct stu x[ ]) //函数定义，结构体数组作参数
{
    int i, n=0;
    for( i=0; i<N; i++)
        if ( x[i].score<60 )    n++;
    return n;
}

int main( )
{
    struct stu st[N];    int num;
    printf("请输入%d个学生的信息: \n", N);
    for(i=0; i<N; i++)
        scanf(" %s%s%d", st[i].num, st[i].name, &st[i].score);
    num=count(st);        //函数调用，数组名作实参
    printf("不及格的学生人数为: %d\n", num);
    return 0;
}
```



7.5.2 指向结构体变量的指针作函数参数

【例7.7】 编写函数按成绩由高到低对学生进行排序，要求用结构体数组名作函数参数。

分析：

学生信息的输入、输出在main函数中进行，编写一个sort函数实现成绩排序。

注意：排序时比较的是学生的成绩(即对结构体变量中的“成绩”成员进行比较)，但交换的是学生的全部信息(即对结构体变量进行交换)。

```
#include<stdio.h>
#define N 10
struct stu
{  char num[20];
   char name[20];
   int score;
};
```



7.5.2 指向结构体变量的指针作函数参数

【例7.7】参考程序

```
void sort( struct stu x[ ] ) //函数定义，结构体数组作形参
{ struct stu temp; //定义一个结构体变量，用作交换时的临时变量
  int i, j, k, m;
  for(i=0; i<N-1; i++)
  { k=i;
    for(j=i+1; j<N; j++)
    { if( x[k].score<x[j].score ) //数组元素的score成员进行比较
      k=j;
    }
    if ( k!=i ) //当k不等于i时，应交换数组元素x[k]和x[i]
    { temp=x[k]; x[k]=x[i]; x[i]=temp; }
  }
}
```



7.5.2 指向结构体变量的指针作函数参数

【例7.7】 参考程序

```
int main( )
{ struct stu st[N];
  int i;
  printf("请输入%d个学生的信息: \n", N);
  for(i=0; i<N; i++)
    scanf(" %s%s%d", st[i].num, st[i].name, &st[i].score);
  sort(st);           //函数调用, 将数组st的首地址传给形参x
  printf("按成绩高低顺序输出学生信息: \n");
  for(i=0; i<N; i++)
    printf(" %s, %s, %d\n", st[i].num, st[i].name, st[i].score);
  return 0;
}
```



7.5.2 指向结构体变量的指针作函数参数

【例7.8】编写函数计算学生的平均分，在主函数中输入学生数据，输出学生的学号、姓名和平均分，**要求用结构指针变量作函数参数。**

```
#include<stdio.h>
#define N 10          //定义符号常量N，表示学生人数
struct student        //定义结构体类型
{ char num[20];
  char name[20];
  int score[3];
  double ave;
};
void average(struct student *p) //结构体指针变量作形参
{ int i, sum=0;
  for(i=0; i<3; i++)
    sum=sum+p->score[i];
  p->ave=sum/3.0;
}
```



7.5.2 指向结构体变量的指针作函数参数

```
int main( )
```

```
{  struct student st[N],*ps;    int i, j;  
  printf("请输入%d个学生的信息:\n", N);  
  for(i=0; i<N; i++)  
  {  scanf(" %s%s", st[i].num, st[i].name);  
    for(j=0; j<3; j++)           //输入3门课的成绩  
      scanf(" %d", &st[i].score[j]);  
  }  
  for(i=0; i<N; i++)           //计算每个学生的平均分  
  {  ps=&st[i];                 //指针变量指向数组元素st[i]  
    average(ps);              //函数调用, 将ps中的地址传给形参p  
  }  
  printf("\n输出学生的平均分:\n");  
  for(i=0; i<N; i++)  
    printf(" %s %s %5.2lf\n", st[i].num, st[i].name, st[i].ave);  
  return 0;  
}
```

7.5.3 函数返回值为结构体类型

◆ 返回结构体类型值的函数定义形式

注意: 结构体类型必须是已经定义好的

结构体类型名 **函数名 (形参表)**
{ 函数体; }

【例7.9】 编写函数实现结构体变量的输入，主函数中通过调用该函数使结构体数组元素得到相应的数据。

```
#include<stdio.h>
#define N 10
struct stu
{   char num[16];
    char name[12];
    double score;
};
```

7.5.3函数返回值为结构体类型

```
struct stu input(void) //注意函数返回值为结构体类型
```

```
{ struct stu b;      //定义一个结构体变量b
```

```
    gets(b.num);
```

```
    gets(b.name);
```

```
    scanf(" %lf", &b.score);    getchar();
```

```
    return b; //返回结构体变量b的值
```

```
}
```

```
int main( )
```

```
{ struct stu a[N]; //定义结构体数组a
```

```
    int i;
```

```
    printf("请按学号、成绩、姓名的顺序输入%d个学生的信息:\n", N);
```

```
    for(i=0; i<N; i++)
```

```
        a[i]=input( ); //调用函数，函数的返回值赋值给数组元素a[i]
```

```
    for(i=0; i<N; i++)
```

```
        printf(" %s, %s, %5.2lf\n" , a[i].num, a[i].name, a[i].score);
```

```
    return 0;
```

```
}
```

7.6 链表

7.6.1 链表的引出

链表是一种常见的动态地进行内存分配的重要数据结构，它可以根据需要随时开辟或释放相应的内存空间。

【问题描述7.2】编程实现以下学生信息的管理工作：(1) 输入 学生信息(各班级人数都不相同)；(2) 出现留级、休学或退学的学生时，从该班的学生信息中将这些学生的信息删除；(3) 出现跳级、插班的学生时，将他们的信息添加到该班的学生信息中。

分析：

第(1)项工作，可以用动态数组来实现。

第(2)项工作，删除某个学生的信息，即删除某个数组元素，有两种方法实现：①将该数组元素置空，②通过移动其后的数组元素将该元素覆盖，但数组的大小并不会改变。

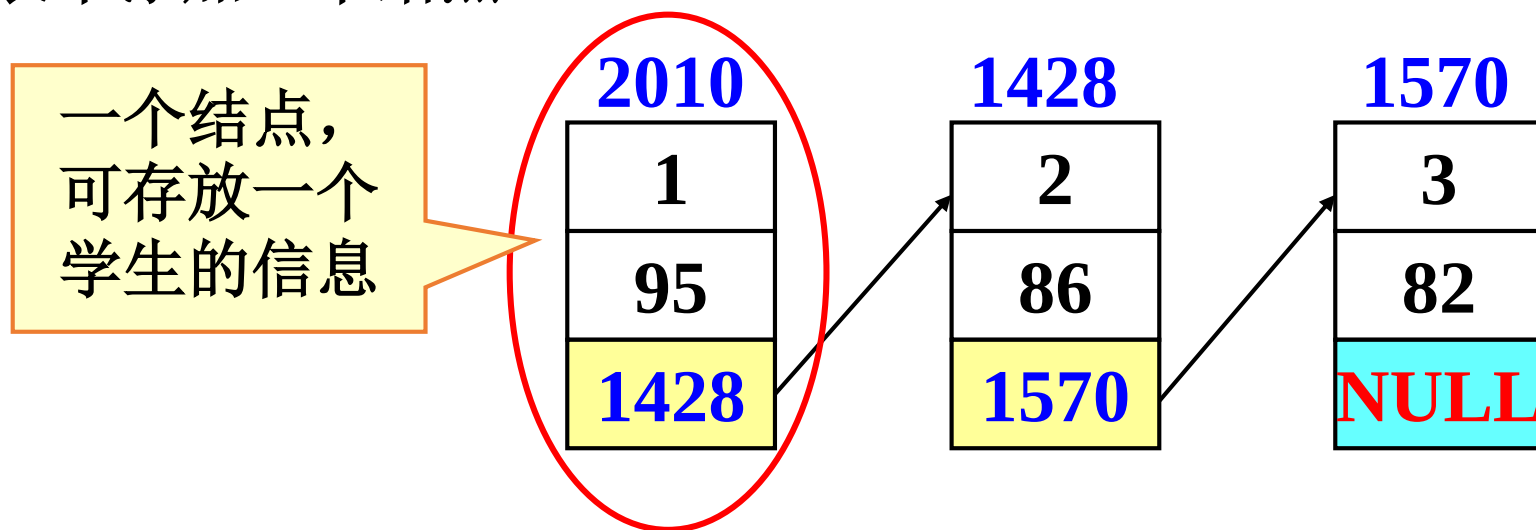
第(3)项工作，添加一个学生的信息，由于数组是“满”的，并没有多余的空间来存放新增加的学生信息，这时只能重新定义一个更大的数组，显然这种方法并不可取。

7.6.1 链表的引出

为解决上述问题，引入链表这种存储结构。

链表是由结点组成的，用户可以根据需要随时添加或删除结点。

一个班级建立一个链表，链表中的一个结点用来存放一个学生的信息，若出现学生留级、休学或退学时，可删去该结点，并释放该结点占用的存储空间；若出现跳级、插班的学生时，则可以在链表中添加一个结点。



7.6.1 链表的引出

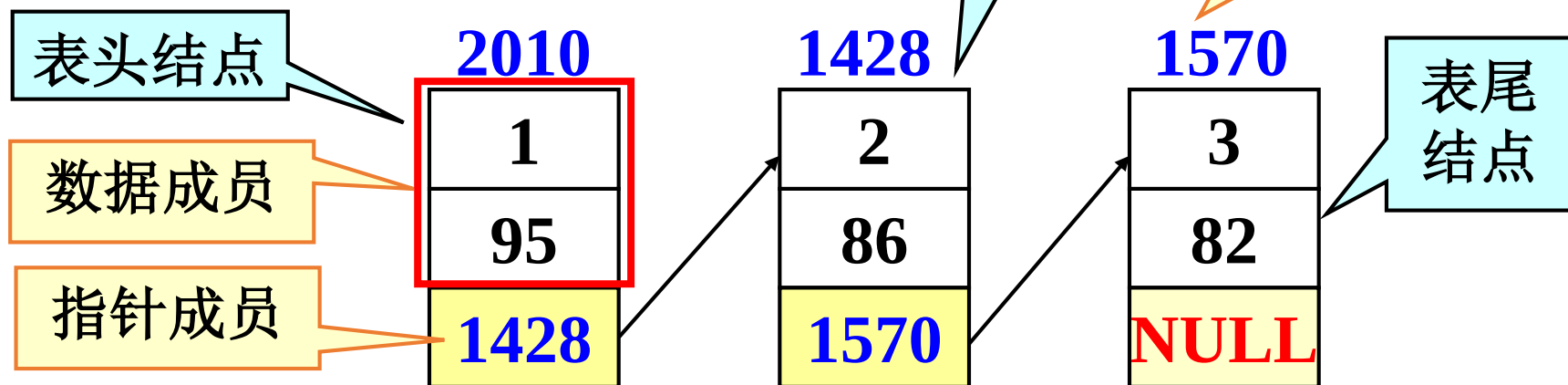
结点是一个结构体类型的数据，每个结点包括两部分，一部分是数据成员，用于存储数据信息；另一部分是指针成员，用于存储下一结点的首地址。

```
struct stu_node
```

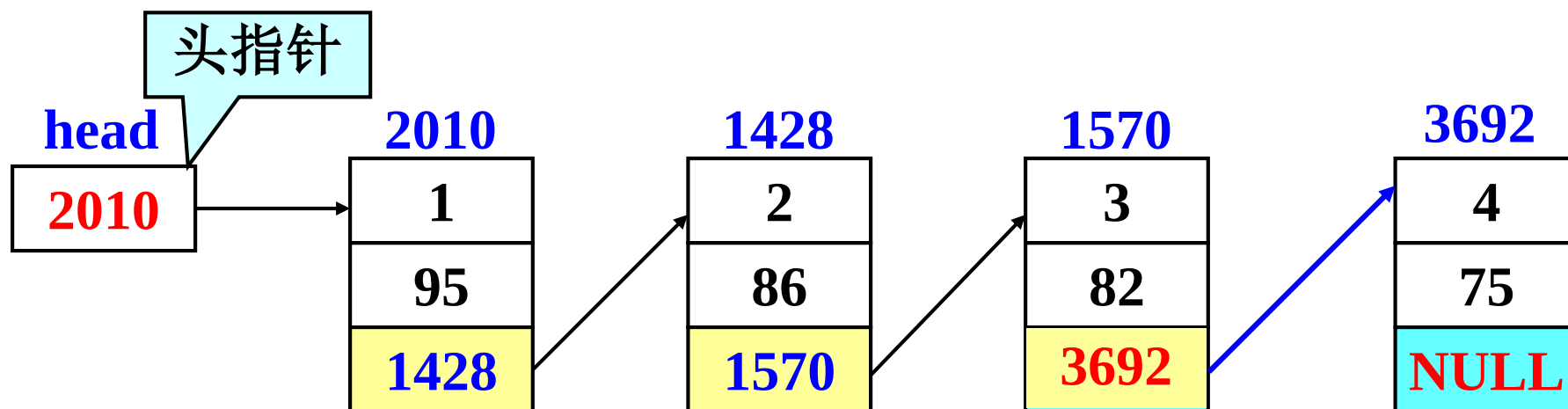
```
{ int num;  
  float score; } 数据成员
```

```
struct stu_node *next; ———> 指针成员
```

```
};
```



7.6.1 链表的引出

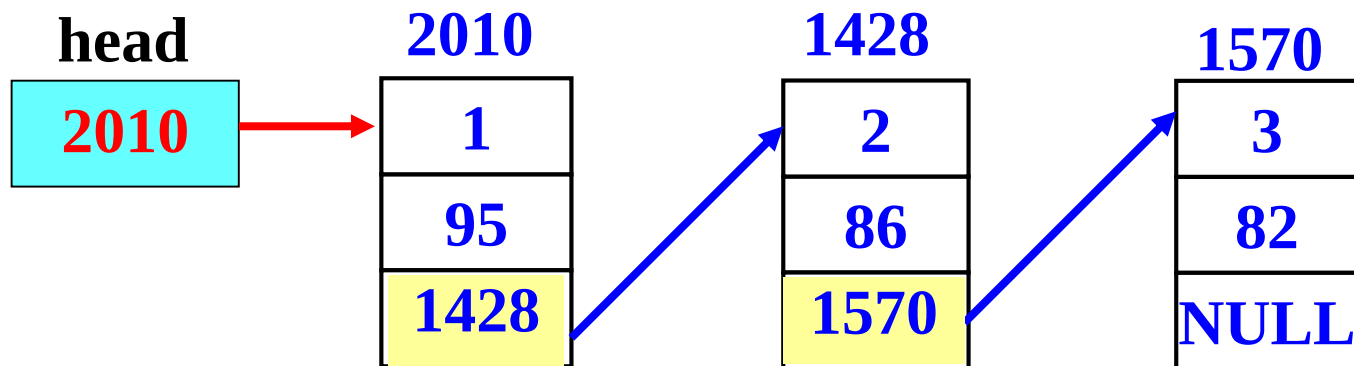


- 链表的每个结点存放在内存中的不同位置，只有找到第1个结点，才能通过第1个结点的指针成员找到第2个结点 ...
因此我们将第1个结点的地址存放在头指针中
- 链表的长度是不固定的，可以随时添加结点，如果将一个结点添加到链表的尾部，则新结点成为表尾结点，它的指针成员必须赋为NULL，而原来的表尾结点则成为中间结点
- 前驱结点与后继结点

7.6.2 链表的建立

◆ 表尾添加法

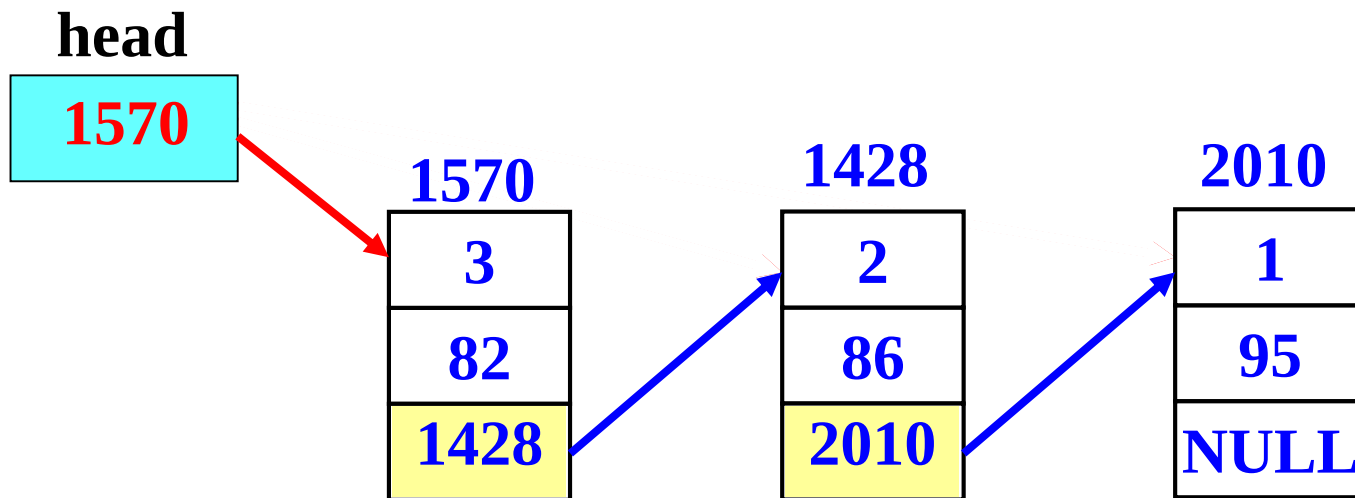
从一个空表开始，将新结点插入到链表的表尾，链表中结点的次序与输入数据的次序是一致的。



7.6.2 链表的建立

◆ 表头添加法

从一个空表开始，将新结点插入到链表的表头，链表中结点的次序与输入数据的次序是**相反**的。



7.6.2 链表的建立

回顾 6.2.2 函数返回指针

◆ 函数的返回值可以是一个指针类型的数据(即地址)

返回指针的函数定义格式:

类型标识符 *函数名(形参列表)
{ 函数体语句; }

说明: 在函数名前加 * , 表明这是一个指针型函数,
函数的返回值是一个指针(即函数返回一个地址)

例: int * fun (int a, int b)
{ 函数体 ; }

7.6.2 链表的建立

回顾 6.2.2 函数返回指针

【例6.6】利用指针函数求两个数中的最大值。

```
#include <stdio.h>
```

```
int *maxp(int *, int *);
```

```
int main( )
```

```
{ int x, y, *p;
```

```
  scanf("%d%d", &x, &y);
```

```
  p=maxp(&x, &y);
```

```
  printf("\nmaxp=%d", *p);
```

```
  return 0;
```

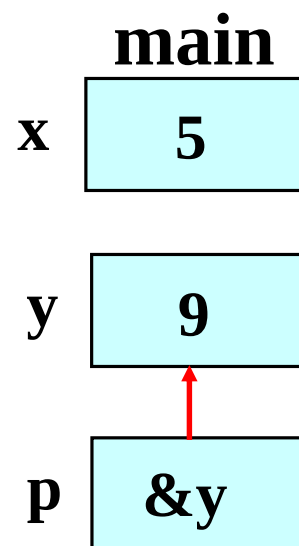
```
}
```

```
int *maxp(int *a, int *b)
```

```
{
```

```
    return *a>*b? a:b;
```

```
}
```



7.6.2 链表的建立

【例7.10】 用表尾添加法编写函数建立一个链表，存放若干名学生的数据，直到输入的学号是 0 为止。

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
struct stu_node
```

定义结构体类型

```
{ int num ;
```

```
float score ;
```

```
struct stu_node *next ;
```

求出结构体类型占用的字节数

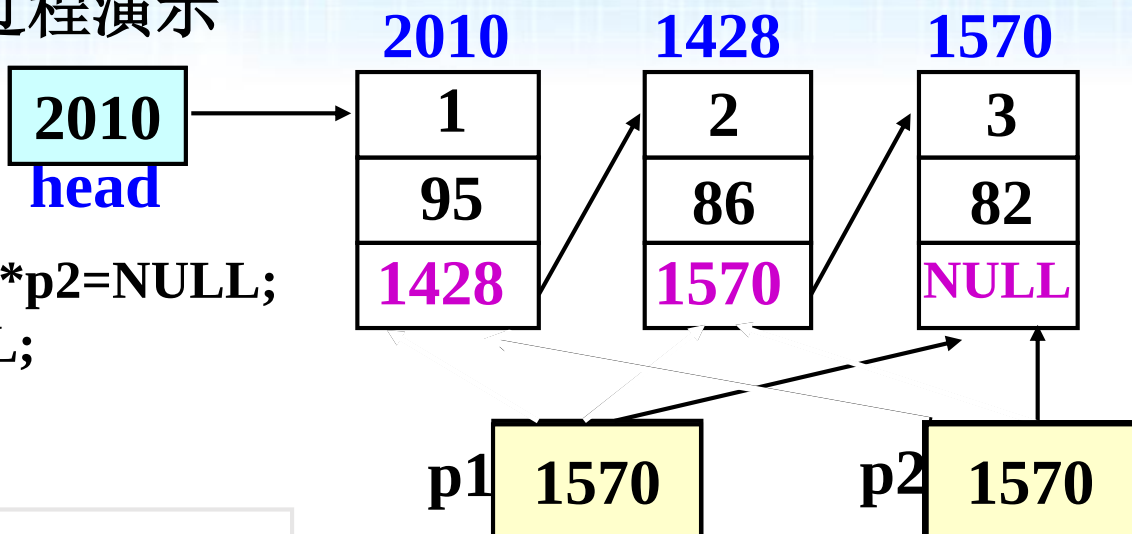
```
};
```

```
#define LEN sizeof(struct stu_node)
```

◆表尾添加法建立链表的过程演示

```

struct stu_node *creat( void )
{  struct stu_node *p1=NULL, *p2=NULL;
   struct stu_node *head=NULL;
   int n;  float s;
   scanf("%d%f", &n, &s);
   while ( n!=0 )
   {  p1=( struct stu_node *) malloc (LEN);
      p1->num=n;  p1->score=s;
      if ( head==NULL ) head=p1;
      else p2->next=p1;
      p2=p1;
      scanf("%d%f", &n, &s);
   }
   p2->next =NULL;
   return(head);
}
    
```



```

while ( n!=0 )
{  p1=( struct stu_node *) malloc (LEN);
   p1->num=n;  p1->score=s;
   p1->next =NULL; //这样写更好
   if ( head==NULL ) head=p1;
   else p2->next=p1;
   p2=p1;
   scanf("%d%f", &n, &s);
}
    
```

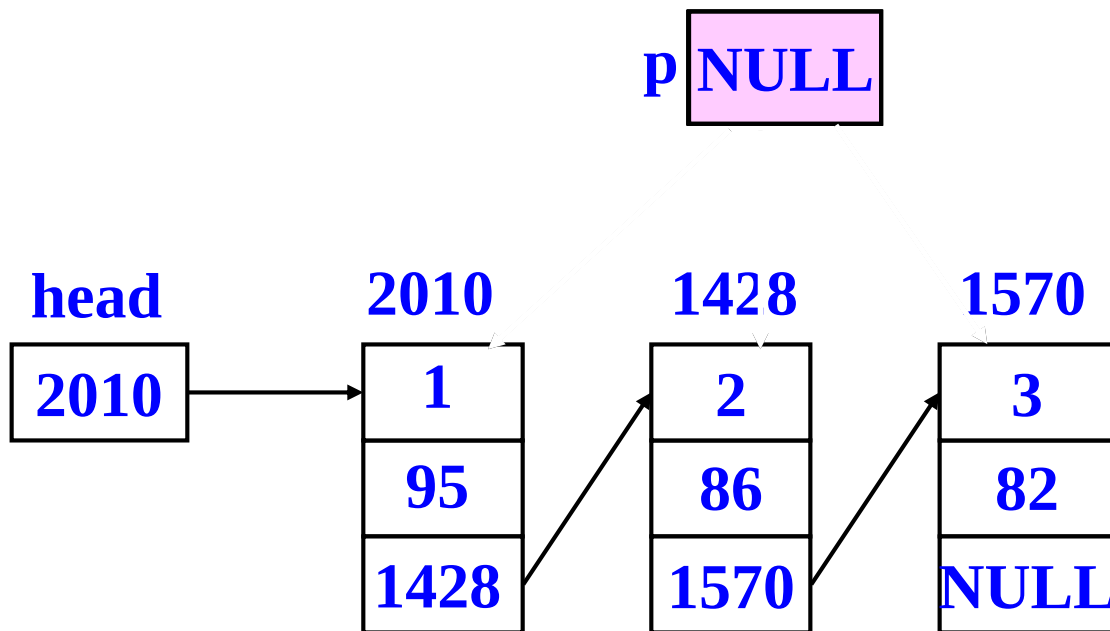
7.6.2 链表的建立

```
struct stu_node *creat( void )    //函数返回值为链表的头指针
{
    struct stu_node *p1=NULL, *p2=NULL, *head=NULL;
    int n; float s;
    scanf(" %d%f", &n, &s);
    while (n!=0)    //判断新结点的学号是否为0
    {
        p1=( struct stu_node *) malloc (LEN); //令p1指向新结点
        p1->num=n;  p1->score=s ;
        if ( head==NULL )    //若头指针为空，则令头指针指向表头结点
            head=p1;
        else
            p2->next=p1;    //否则令当前表尾结点指针成员指向新结点
            p2=p1;          //p2指向新的表尾结点
            scanf(" %d%f", &n, &s) ;
    }
    p2->next =NULL;        //表尾结点的指针成员赋空值
    return(head);         //返回头指针
}
```

7.6.3 链表的输出

◆ 链表的输出(遍历)

让指针变量依次指向链表中的各个结点，并输出其数据成员的值。



输出:
1, 95
2, 86
3, 82

7.6.3 链表的输出

【例7.11】编写函数输出链表中所有结点的数据成员的值。

```
void list(struct stu_node *head)
{
    struct stu_node *p;
    if (head==NULL) printf("链表为空! \n");
    else
    {
        printf("链表信息如下:\n");
        p=head;           //p赋值为head, 即p指向表头结点
        while (p!=NULL)
        {
            printf(" %d, %5.2f\n", p->num, p->score);
            p=p->next;     //令p指向下一个结点
        }
    }
}

int main( )
{
    struct stu_node *head;
    head=creat( );       //建立一个链表, 该链表的头指针赋给head
    list(head);           //输出head所指向的链表
    return 0;
}
```

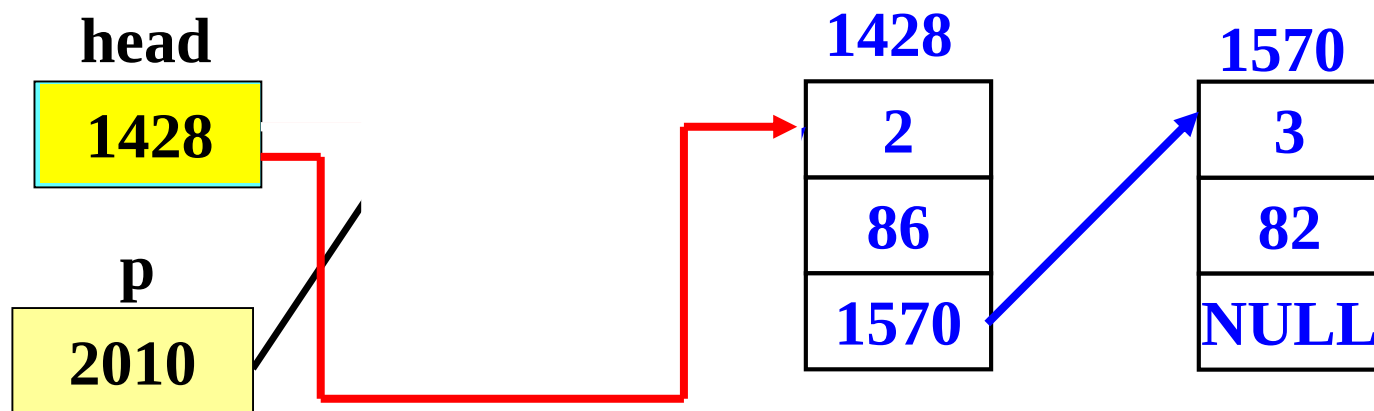


7.6.4 链表的删除操作

◆ 链表的删除操作

将链表中的某个指定的结点从链表中分离出来，不再与其他结点有任何联系，并且释放已删除结点所占据的内存空间。

(1) 删除表头结点：让头指针指向链表的第2个结点



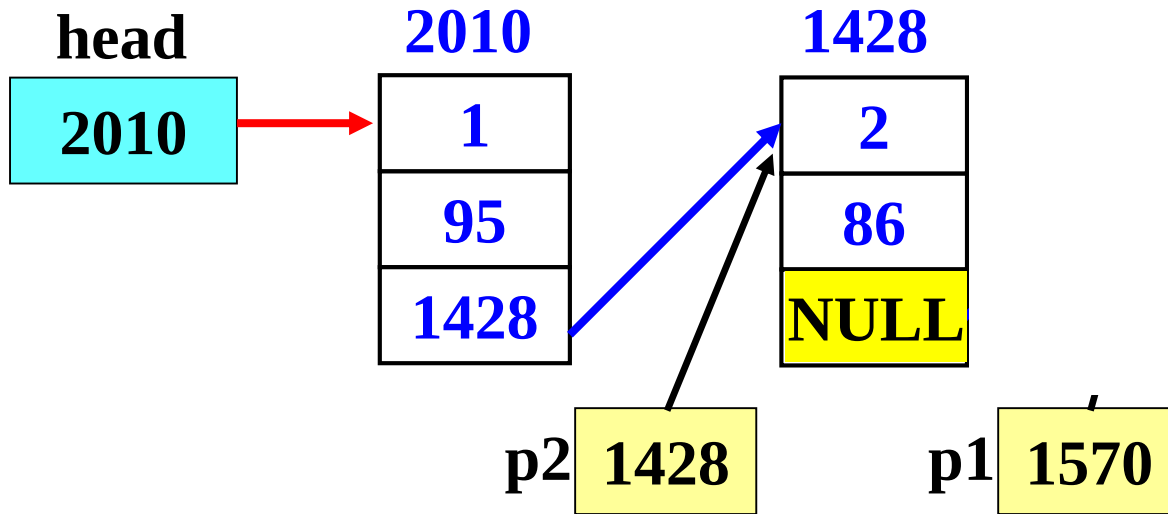
step1: 让指针变量p指向要删除的结点即表头结点 **p=head;**

step2: 重新给头指针赋值，使它指向第2个结点 **head=p->next;**

step3: 释放删除的结点空间 **free(p);**

7.6.4 链表的删除操作

(2) 删除表尾结点：将链表倒数第2个结点的指针成员赋为NULL



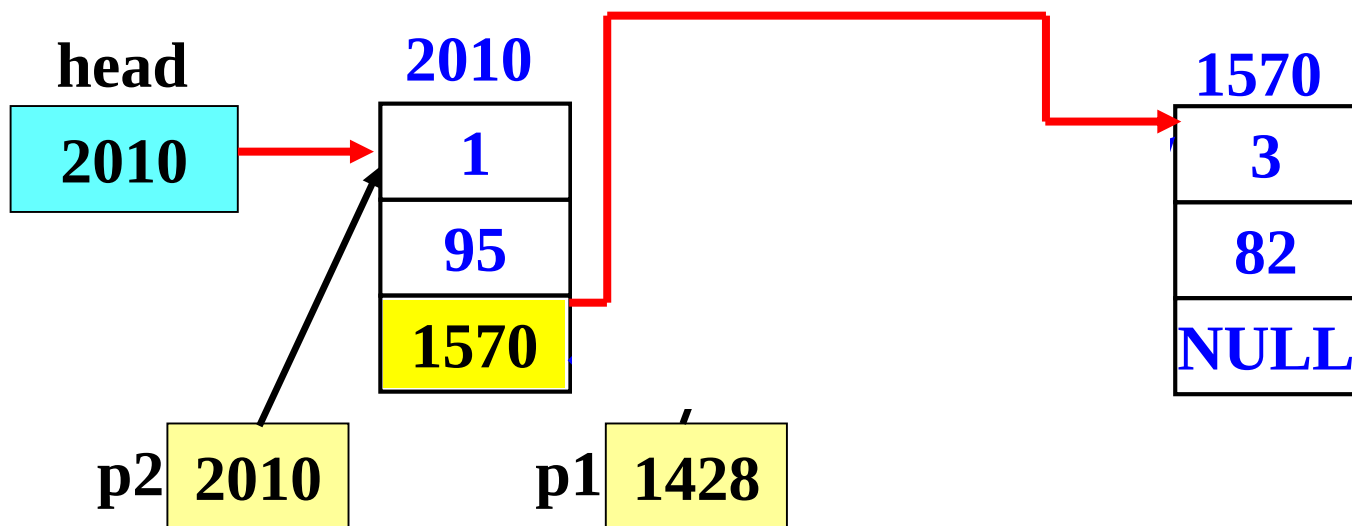
step1: 让指针变量p1指向要删除的结点即表尾结点
让指针变量p2指向要删除结点的前一个结点

step2: 将删除结点的前驱结点的指针成员赋空值 **p2->next=NULL;**

step3: 释放删除的结点空间 **free(p1);**

7.6.4 链表的删除操作

(3) 删除中间结点：让要删除结点的前驱结点指向要删除结点的后继结点



step1: 让p1指向要删除的结点, 让p2指向要删除结点的前驱结点

step2: 前驱结点的指针成员赋为要删除结点的指针成员值

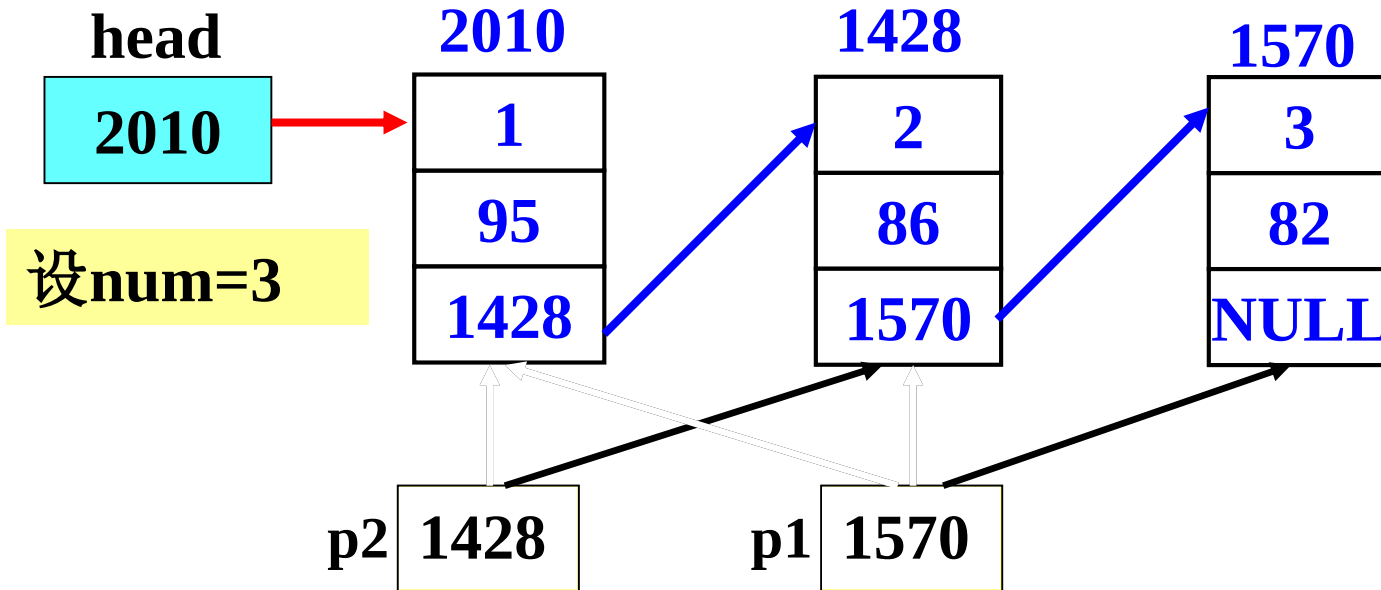
p2->next=p1->next;

step3: 释放删除的结点空间 **free(p1);**

7.6.4 链表的删除操作

- 让p1指向要删除的结点，让p2指向其前驱结点的过程演示

```
struct stu_node *p1, *p2=NULL;  
p1=head;  
while( (num!=p1->num) && (p1->next!=NULL) )  
    { p2=p1;    p1=p1->next; }
```



7.6.4 链表的删除操作

```
struct stu_node *del (struct stu_node *head, int num)
{
    struct stu_node *p1, *p2=NULL;
    if (head==NULL) printf("链表为空不能进行删除操作!\n");
    else
    {
        p1=head;                //p1指向表头结点
        while ((num!=p1->num) && (p1->next!=NULL))
        {
            p2=p1;                //p2指向p1所指向的结点
            p1=p1->next;          //p1指向下一个结点
        }
        if ( num==p1->num ) //判断p1是否指向要删除的结点
        {
            if (p1==head) head=p1->next; //删除表头结点
            else p2->next=p1->next;      //删除中间或表尾结点
            free(p1);                    //释放已删除的结点空间
            printf("结点已删除!\n");
        }
        else printf("链表中不存在该结点, 不能删除!\n");
    }
    return( head );
}
```

7.6.4 链表的删除操作

- 通过使用循环, 可以不断删除结点, 直到用户输入字符' n'为止。

```
void main( )
```

```
{ struct stu_node *head;  int delnum;
```

```
  char ch='y';
```

```
  head=creat( );    list(head);
```

```
  printf(“进行删除结点操作\n”);
```

```
  while(ch=='y' || ch=='Y')
```

```
{  printf(“输入要删除学生的学号: ” );
```

```
  scanf(“%d”, &delnum);  getchar( );
```

```
  head=del (head, delnum);
```

```
  printf(“继续删除其他学生的信息吗? (y/n): ” );
```

```
  ch=getchar( );           //读取字符' y'或字符' n'
```

```
  getchar( );             //空读, 读取回车符
```

```
}
```

```
printf(“删除后的学生信息: \n”);
```

```
list(head);
```

```
}
```

7.6.5 链表的插入操作

◆ 链表的插入操作

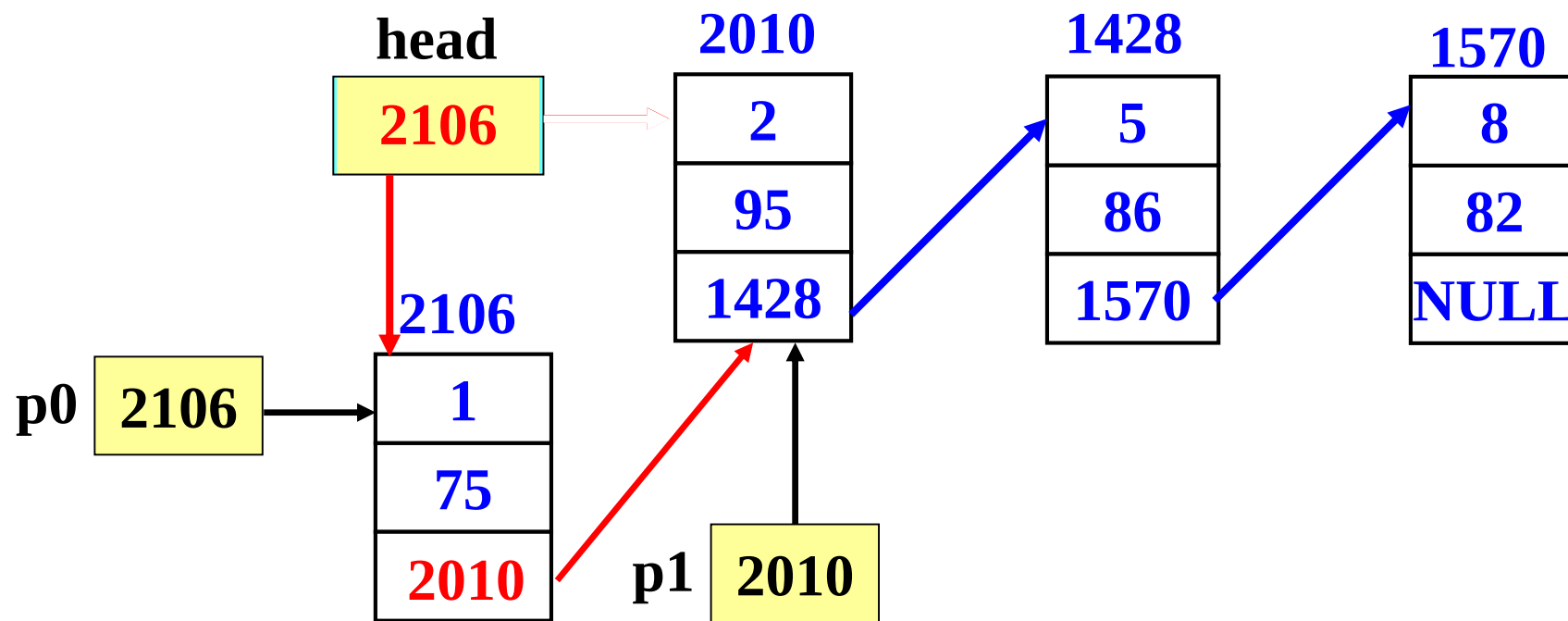
将一个新结点插入到一个已有链表的适当位置。

(1) 插入的结点作表头结点

让p0指向新结点即要插入的结点，

让p1 指向插入位置上的结点即表头结点

head = p0;
p0->next = p1;

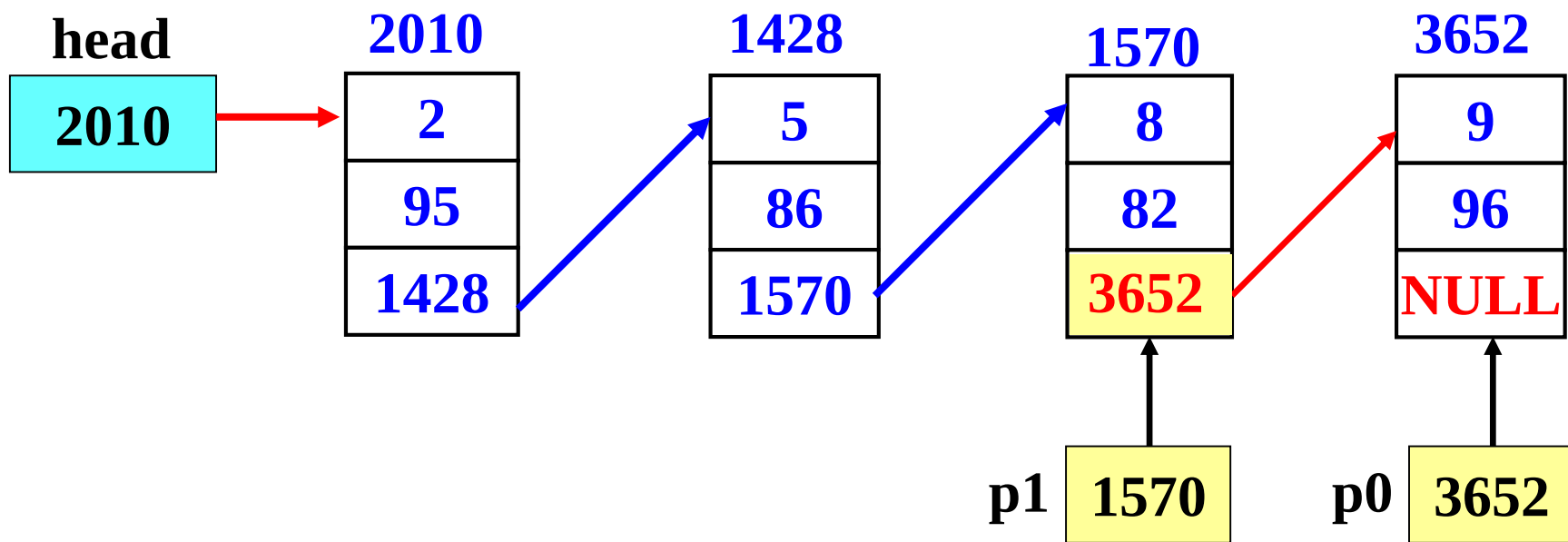


7.6.5 链表的插入操作

(2) 插入的结点作表尾结点

让p0指向新结点即要插入的结点，
让p1 指向插入位置上的结点即表尾结点

p1->next = p0;
p0->next = NULL;



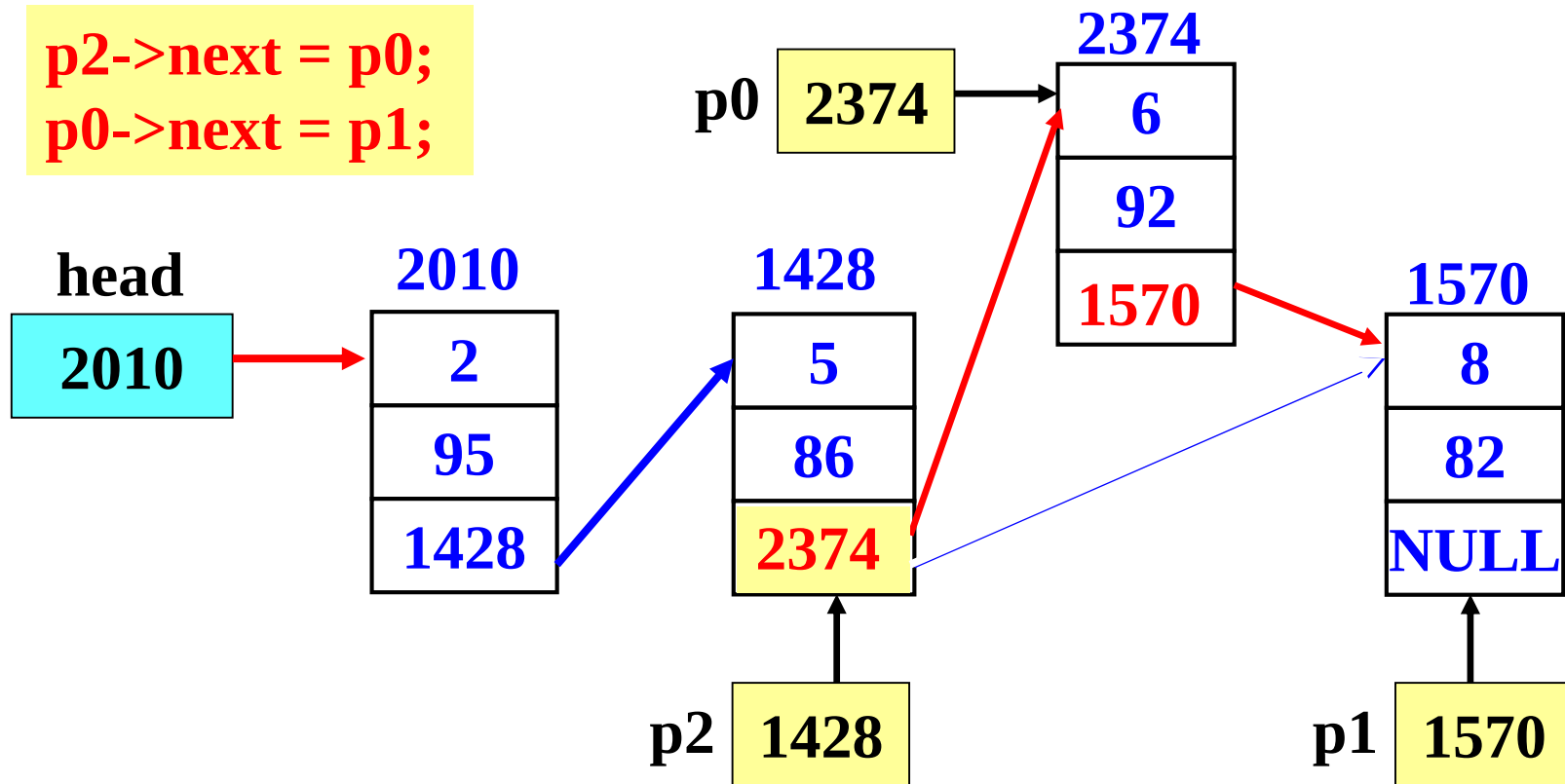
7.6.5 链表的插入操作

(3)插入的结点作中间结点

让p0指向新结点即要插入的结点，

让p1 指向插入位置上的结点， p2指向p1的前驱结点

$p2 \rightarrow next = p0;$
 $p0 \rightarrow next = p1;$



【例7.13】编写函数实现向有序链表中插入一个结点。

```
struct stu_node *insert ( struct stu_node *head )
{ struct stu_node *p0 , *p1 , *p2 ;
  p0=( struct stu_node *) malloc (LEN); //p0指向产生的新结点
  p0->next = NULL; //p0指针成员赋NULL
  printf("输入学号和成绩: ");
  scanf ("%d%f", &p0->num, &p0->score); //输入数据成员
  getchar(); //空读, 读取回车符
  if (head==NULL) head = p0; //链表为空时, 令head指向新结点
  else //链表不为空时, 执行以下插入操作
  { p1=head; //p1 指向表头结点
    while ((p0->num>p1->num) && (p1->next!=NULL))
    { p2 = p1; p1 = p1->next; }
    if ( p0->num<=p1->num )
    { if ( head==p1) head = p0; //插入表头结点
      else p2->next=p0; //插入中间结点
        p0->next = p1; // p0的指针成员指向p1
      }
    else p1->next=p0; //插入表尾结点
  }
  return( head );
}
```



7.6.5 链表的插入操作

● 插入结点过程演示

(1) 插入链表的第一个结点

```
struct stu_node *insert ( struct stu_node *head )
```

```
{ struct stu_node *p0 , *p1 , *p2 ;
```

```
  p0=( struct stu_node *) malloc (LEN);
```

```
  p0->next = NULL;
```

```
  printf("输入学号和成绩: ");
```

```
  scanf ("%d%f", &p0->num, &p0->score);
```

```
  getchar();
```

```
  if (head==NULL) head = p0;
```

```
  else
```

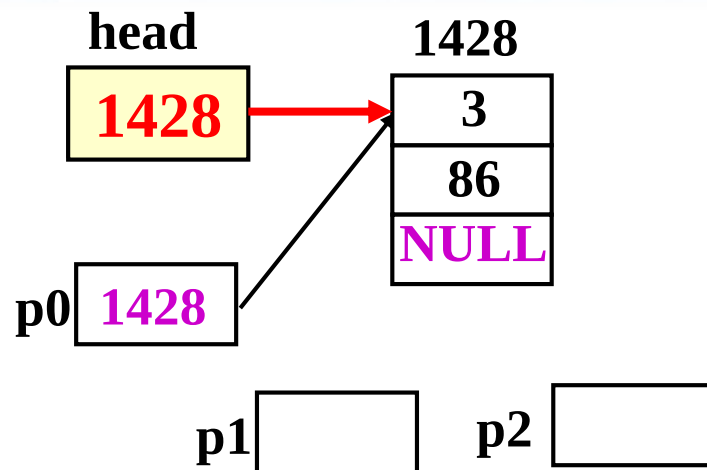
```
  {
```

```
    ... //这部分语句不执行
```

```
  }
```

```
  return( head ) ;
```

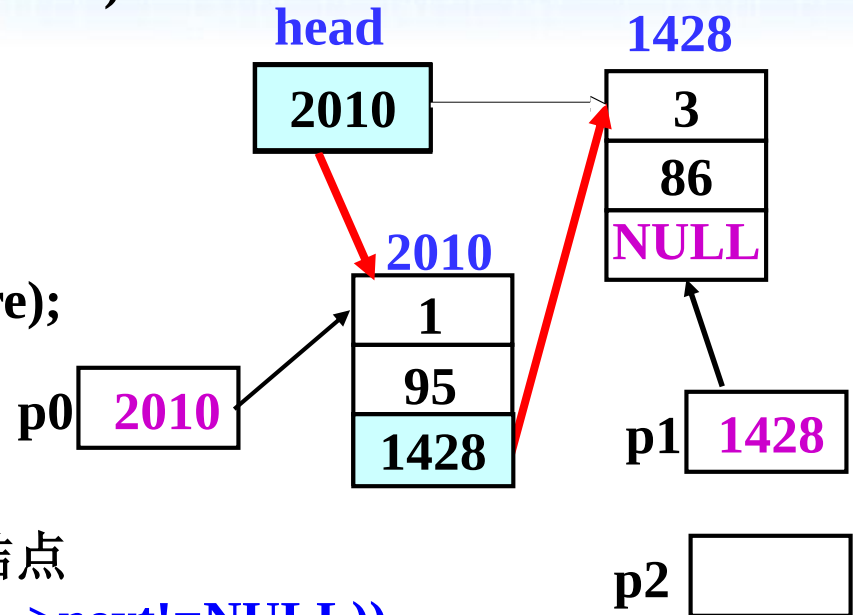
```
}
```



(2) 插入结点作表头结点

```
struct stu_node *insert ( struct stu_node *head )
```

```
{ struct stu_node *p0 , *p1 , *p2 ;  
  p0=( struct stu_node *) malloc (LEN);  
  p0->next = NULL;  
  printf("输入学号和成绩: ");  
  scanf ("%d%f", &p0->num, &p0->score);  
  getchar();  
  if (head==NULL) head = p0;  
  else  
  { p1=head;           //p1 指向表头结点  
    while ((p0->num>p1->num) && (p1->next!=NULL))  
    { p2 = p1;    p1 = p1->next;    }  
    if ( p0->num<=p1->num )  
    { if ( head==p1) head = p0; //插入表头结点  
      else p2->next=p0;  
      p0->next = p1;  
    }  
    else p1->next=p0;  
  }  
  return( head ) ;  
}
```



(2) 插入结点作表尾结点

```
struct stu_node *insert ( struct stu_node *head )
```

```
{ struct stu_node *p0 , *p1 , *p2 ;  
  p0=( struct stu_node *) malloc (LEN);  
  p0->next = NULL;  
  printf("输入学号和成绩: ");  
  scanf ("%d%f", &p0->num, &p0->score);  
  getchar();  
  if (head==NULL) head = p0;  
  else
```

```
{ p1=head;           //p1 指向表头结点
```

```
while ((p0->num>p1->num) && (p1->next!=NULL))
```

```
{  p2 = p1;    p1 = p1->next;  }
```

```
if ( p0->num<=p1->num )
```

```
{  if ( head==p1) head = p0;
```

```
else    p2->next = p0;
```

```
p0->next = p1;
```

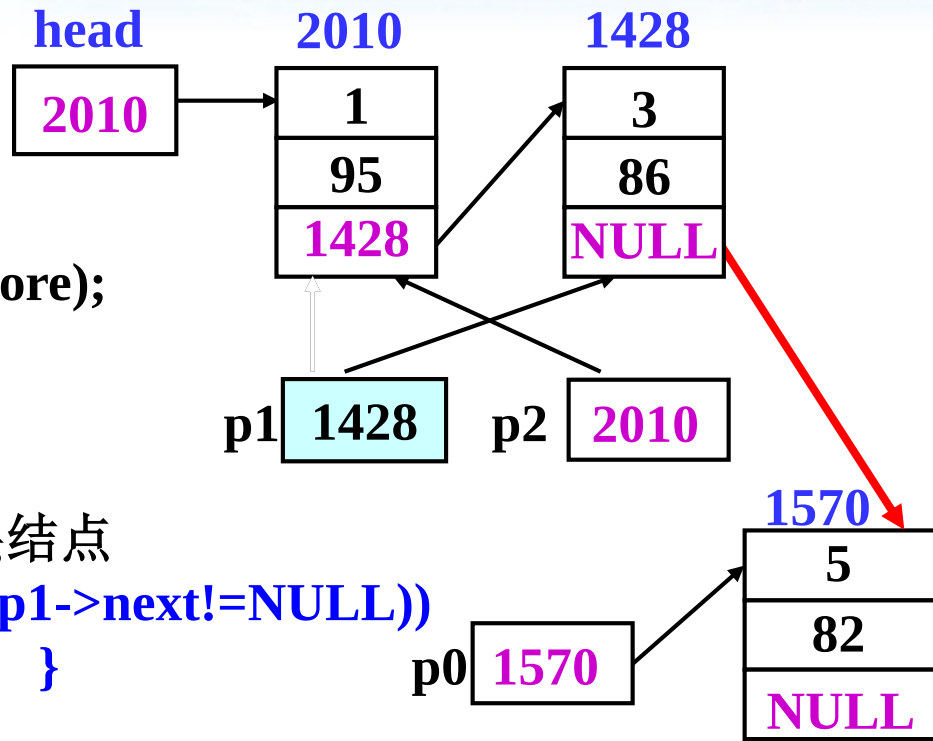
}

```
else p1->next=p0;
```

}

```
return( head ) ;
```

}



7.6.5 链表的插入操作

- 链表为空时, 通过多次调用insert函数, 可以建立一个有序链表

```
void main( )
```

```
{ struct stu_node *head;
```

```
  char ch='y';
```

```
  head=creat( );
```

```
  list(head);
```

```
  printf("进行插入结点操作\n");
```

```
  while(ch=='y' || ch=='Y')
```

```
  { head=insert(head);           //调用插入结点函数
```

```
    printf("继续插入学生信息吗? (y/n): ");
```

```
    ch=getchar( );               //读取字符' y'或字符' n'
```

```
    getchar( );                  //空读
```

```
  }
```

```
  printf("插入后的学生信息: \n");
```

```
  list(head);
```

```
}
```



培养创新思维 提升编程能力